

UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y
SISTEMAS

ESCUELA DE INGENIERÍA DE SISTEMAS



Trabajo Encargado N° 4

ANÁLISIS DE COMPORTAMIENTO DE HARDWARE Y SOFTWARE EN ENTORNO LINUX

CURSO:

SISTEMAS OPERATIVOS

PRESENTADO POR:

AQUISE ZEA KARLOS JOEL

CANAZAS CHOQUEMAMANI ELVIS CRISTIAN

DOCENTE:

RUELAS ACERO DONIA ALIZANDRA

PUNO – PERÚ

2026

Repositorio GitHub:

<https://github.com/Cristian455766/Tarea4-SistemasOperativos>

Índice

1. Objetivo	2
2. Entorno de pruebas	2
3. Explicación del script de automatización (TAREA.sh)	2
4. Resultados y análisis de hardware	3
4.1. Procesador (CPU)	3
4.2. Memoria RAM	3
4.3. Almacenamiento (Disco)	3
4.4. Tarjeta gráfica (GPU)	3
5. Resultados y análisis de software	6
5.1. Software instalado	6
5.2. Programas de usuario	6
5.3. Entorno gráfico	7
5.4. Procesos de sistema	7
6. Conclusión	7

1. Objetivo

El presente informe tiene como propósito documentar la creación y ejecución de un script de monitoreo automatizado en el sistema operativo Ubuntu (distribución Linux seleccionada en la Tarea 3), con el fin de analizar el comportamiento del hardware asignado, el consumo de recursos de los procesos lógicos, los programas en ejecución y la cantidad de software instalado en el sistema.

2. Entorno de pruebas

Las pruebas se realizaron bajo un entorno virtualizado utilizando el software VirtualBox. Se ejecutó una sesión “Live” (*Try Ubuntu*) para evitar modificaciones permanentes en el almacenamiento del equipo host. El script desarrollado en lenguaje Bash (`TAREA.sh`) integra comandos nativos de diagnóstico del sistema operativo.

3. Explicación del script de automatización (TAREA.sh)

Un script es un archivo de texto que contiene una secuencia de comandos que el sistema operativo ejecuta de forma automatizada. Para esta práctica, se desarrolló un script en Bash que evita la ejecución manual de múltiples comandos de diagnóstico. A continuación se detalla la función de cada línea del código utilizado:

- `#!/bin/bash`: Conocido como *shebang*, indica al sistema operativo que debe utilizar el intérprete de comandos Bash para ejecutar el archivo.
- `echo`: Imprime texto en la consola. Se usó para generar los títulos y separar visualmente las métricas.
- `lscpu | grep "Model name"`: El comando `lscpu` lista la arquitectura del procesador, y `grep` filtra para mostrar únicamente el nombre del modelo exacto del CPU.
- `top -bn1 | grep Cpu(s)`: Toma una muestra instantánea (`-bn1`) del rendimiento general del sistema y extrae el porcentaje de uso e inactividad del procesador.
- `free -h`: Muestra el estado de la memoria RAM. El parámetro `-h` (*human-readable*) formatea los datos en Gigabytes (Gi) o Megabytes (Mi).
- `df -h /`: Analiza el sistema de archivos y muestra la capacidad, el espacio usado y el espacio disponible del directorio raíz (/).
- `dpkg -l | wc -l`: Cuenta la cantidad total de **programas instalados** en el sistema (paquetes registrados por el gestor `dpkg`).
- `ps -e -no-headers | wc -l`: Cuenta la cantidad de **programas en ejecución** (procesos activos) en el sistema en ese instante.

- `ps -eo pid,cmd,%mem,%cpu -sort=-%cpu | head -n 6`: Comando compuesto donde `ps` lista los procesos actuales, `-sort=-%cpu` los ordena de mayor a menor consumo de procesador, y `head -n 6` recorta la salida para mostrar los 5 procesos principales.

4. Resultados y análisis de hardware

La ejecución del script arrojó la siguiente información sobre los recursos físicos y virtuales:

4.1 Procesador (CPU)

El sistema operativo virtualizado reconoce correctamente la arquitectura del equipo host, identificando un procesador *12th Gen Intel(R) Core(TM) i7-12700KF*. Durante la muestra, el procesador registró un 97.8 % de inactividad (`id`), evidenciando un rendimiento óptimo sin cuellos de botella.

4.2 Memoria RAM

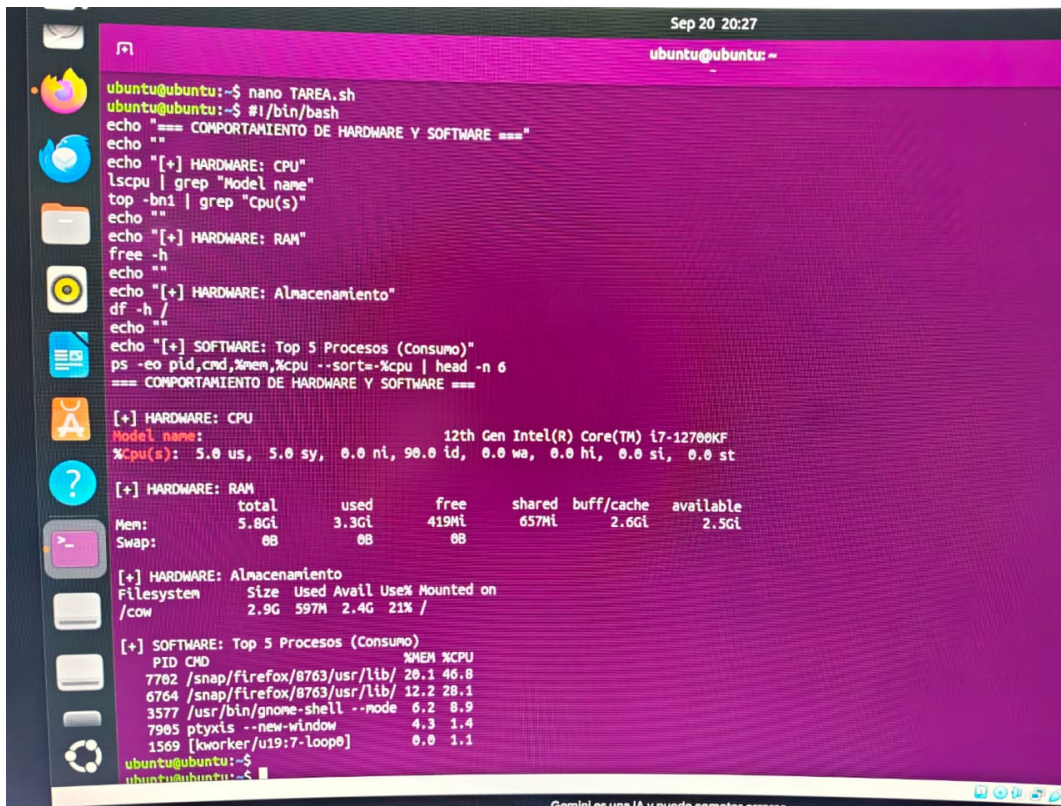
El sistema detecta un total de 5.8Gi asignados a la máquina virtual. Se encuentran en uso 3.7Gi, dejando una memoria disponible de 2.1Gi para cargas futuras.

4.3 Almacenamiento (Disco)

La partición raíz (`/cow`) cuenta con una capacidad total de 2.9G. Actualmente se ha utilizado el 26 % (744M), manteniendo libres 2.2G.

4.4 Tarjeta gráfica (GPU)

Al operar en VirtualBox, el sistema interactúa con un controlador de video virtualizado responsable de renderizar el entorno de escritorio, delegando la carga gráfica de manera estable sin saturar el procesador principal.



```
Sep 20 20:27
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ nano TAREA.sh
ubuntu@ubuntu:~$ #!/bin/bash
echo "=== COMPORTAMIENTO DE HARDWARE Y SOFTWARE ==="
echo ""
echo "[+] HARDWARE: CPU"
lscpu | grep "Model name"
top -bn1 | grep "Cpu(s)"
echo ""
echo "[+] HARDWARE: RAM"
free -h
echo ""
echo "[+] HARDWARE: Almacenamiento"
df -h /
echo ""
echo "[+] SOFTWARE: Top 5 Procesos (Consumo)"
ps -eo pid,cmd,%mem,%cpu --sort=-%cpu | head -n 6
echo "=== COMPORTAMIENTO DE HARDWARE Y SOFTWARE ==="

[+] HARDWARE: CPU
Model name:                  12th Gen Intel(R) Core(TM) i7-12700KF
%Cpu(s):  5.0 us,  5.0 sy,  0.0 ni, 90.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st

[+] HARDWARE: RAM
              total        used        free      shared  buff/cache   available
Mem:          5.8Gi        3.3Gi        419Mi        657Mi        2.6Gi        2.5Gi
Swap:          0B           0B           0B

[+] HARDWARE: Almacenamiento
Filesystem      Size  Used Avail Use% Mounted on
/cow             2.9G  597M  2.4G  21% /

[+] SOFTWARE: Top 5 Procesos (Consumo)
  PID CMD                                %MEM %CPU
 7702 /snap/firefox/8763/usr/lib/ 20.1 46.8
 6764 /snap/firefox/8763/usr/lib/ 12.2 28.1
 3577 /usr/bin/gnome-shell --mode 6.2 8.9
 7905 ptexis --new-window          4.3 1.4
 1569 [kworker/u19:7-loop0]         0.0 1.1

ubuntu@ubuntu:~$
```

Figura 1: Primera ejecución del script TAREA.sh en Ubuntu (VirtualBox).



```
Sep 22 00:11
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ nano TAREA.sh
ubuntu@ubuntu:~$ bash TAREA.sh
=== COMPORTAMIENTO DE HARDWARE Y SOFTWARE ===

[+] HARDWARE: CPU
Model name: 12th Gen Intel(R) Core(TM) i7-12700KF
%Cpu(s): 2.4 us, 2.4 sy, 0.0 ni, 95.1 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st

[+] HARDWARE: RAM
Mem: total used free shared buff/cache available
Swap: 0B 0B 0B 0B 0B 0B

[+] HARDWARE: Almacenamiento
Filesystem Size Used Avail Use% Mounted on
/cow 2.9G 643M 2.3G 22% /

[+] SOFTWARE: Top 5 Procesos (Consumo)
PID CMD MEM% %CPU
6912 /snap/firefox/8763/usr/lib/ 22.0 43.5
6125 /snap/firefox/8763/usr/lib/ 11.8 25.6
3621 /usr/bin/gnome-shell --mode 6.1 6.6
6285 [kworker/u18:0-events_unbou 0.0 0.8
7267 /snap/firefox/8763/usr/lib/ 1.8 0.7

ubuntu@ubuntu:~$
```

Figura 2: Segunda ejecución del script TAREA.sh: diagnóstico de hardware y Top 5 de procesos con mayor consumo.

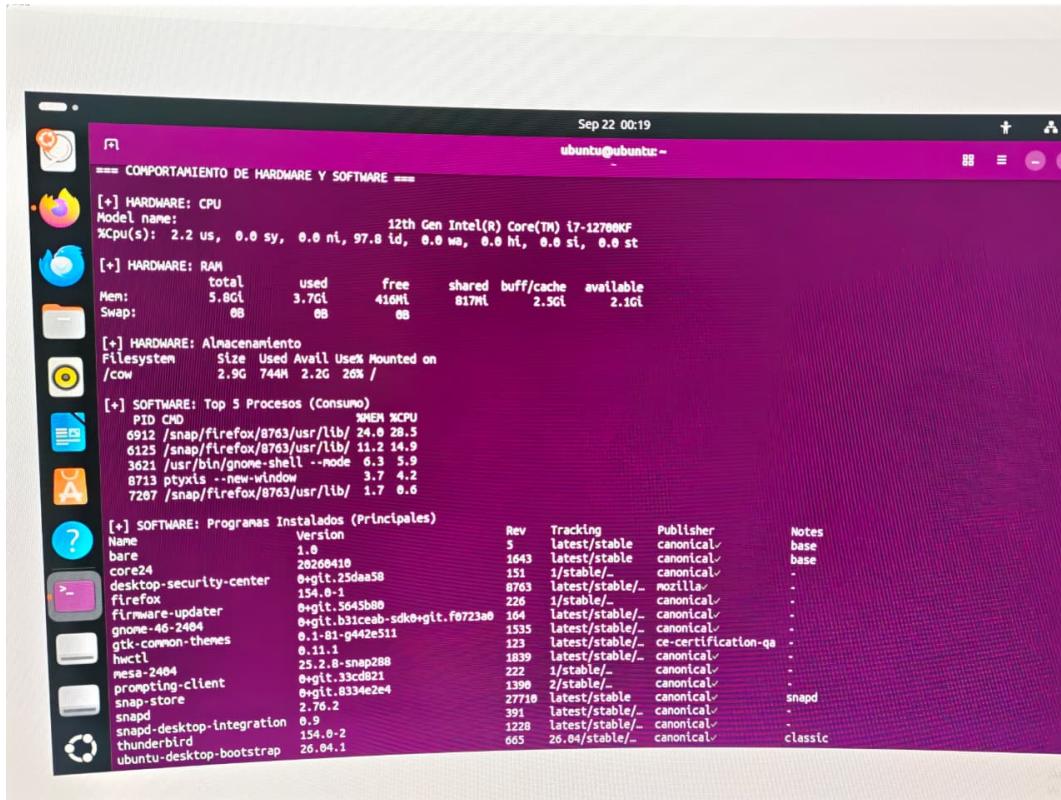


Figura 3: Tercera ejecución del script `TAREA.sh`: listado de programas instalados en Ubuntu.

5. Resultados y análisis de software

El script permitió identificar tanto los **programas en ejecución** con su consumo de recursos, como la cantidad de **software instalado** en el sistema operativo.

5.1 Software instalado

Mediante el comando `dpkg -l | wc -l` se obtuvo el número total de paquetes instalados en Ubuntu. En la tercera ejecución (Figura 3) se listan algunos de los programas principales instalados, como `base`, `core24`, `firefox`, `gnome-46-2404`, `snap-store`, `thunderbird`, entre otros, todos gestionados desde el repositorio oficial *canonical*.

5.2 Programas de usuario

El programa líder en consumo es el navegador web Firefox. Sus procesos principales (PID 6912, 6125 y 7207) exigen combinados más del 35 % de la memoria RAM y generan picos de uso de CPU del 28.5 %, 14.9 % y 0.6 % respectivamente.

5.3 Entorno gráfico

Se encuentra `gnome-shell` (PID 3621), utilizando un 6.3 % de la RAM y un 5.9 % del procesador. Es un consumo base esperado para la gestión de ventanas de la interfaz gráfica de Ubuntu.

5.4 Procesos de sistema

Las herramientas del núcleo tienen un impacto mínimo, como el proceso `kworker` (PID 6285) con un consumo nulo de RAM y 0.8 % de CPU, demostrando la eficiencia del kernel de Linux.

6. Conclusión

El entorno virtualizado Ubuntu administra eficientemente el hardware asignado. La automatización mediante el script Bash confirma que no existe saturación del procesador, manteniendo una amplia capacidad de procesamiento disponible. El script logró identificar correctamente la cantidad de software instalado y los programas en ejecución con su consumo de recursos. El mayor impacto en el consumo de RAM y CPU recae exclusivamente en la ejecución a nivel de usuario (Firefox) y el renderizado del entorno gráfico, comprobando la estabilidad estructural del sistema operativo frente a cargas de trabajo cotidianas.